TradeOps

Milestone 3: Conceptual Multidimensional Model, ROLAP Implementation, and OLAP Analysis

Item	Details
Course	IE6750 Data Warehousing & Integration, Northeastern University
Assignment	Milestone 3: MultiDim conceptual model, relational (ROLAP) model, primary events, OLAP operations
Group Number	Group 08
Member	Jesus D. Jimenez Ballestas
Project Title	TradeOps: A Data Warehouse for Prop-Firm Futures Trading Operations
Data Source	Operational database from Milestone 2 (synthetic, seeded data with real contract specifications)
Database	PostgreSQL 16, database tradeops: schema tradeops (operational) and schema tradeops_dw (warehouse)
Repository	https://github.com/Jesus-D-Jimenez/tradeops
Date	October 2026

1. Project Summary

TradeOps was developed for a futures trading business that uses capital provided by proprietary trading firms, often referred to as prop firms. The operation purchases evaluation accounts, executes trades using a leader account, replicates those trades across follower accounts, and requests payouts after accounts become funded. Milestones 1 and 2 established the operational database for these activities. In this milestone, the operational data is structured as a warehouse for reporting and analysis. This stage covers a conceptual model using MultiDim notation, its conversion into a relational (ROLAP) model, the PostgreSQL implementation and data-loading process, and nine OLAP analyses tied to the project's business questions. The findings presented here are based on executing the provided SQL scripts against the warehouse.

2. Problem Definition

Problem statement. Determining the operation's actual return is difficult because evaluation, reset, activation, and renewal fees are stored separately from trading profits and payouts. Another challenge is determining which strategies, trading sessions, and instruments are linked to account-ending rule violations or to calculate the extra costs caused by copying trades. By integrating these records through shared dimensions, the warehouse allows these questions to be answered through analytical queries without manually reconciling separate reports.

The warehouse addresses the same five business questions defined in Milestone 2:

- **BQ1.** What is the operation's net return, calculated as payouts received minus all fees, by firm, plan, and period?
- **BQ2.** Which strategies and instruments generate the most net P&L, and which ones are associated with the most rule violations?
- **BQ3.** How does trading performance differ by session and on major economic event days such as CPI, NFP, and FOMC?
- **BQ4.** What percentage of evaluations are passed, and how long does it take to pass them?
- **BQ5.** How much additional slippage is created when trades are copied from the leader account?

Milestone 3 Revisions

For this milestone, the assignment allows updates to the problem statement, operational database, and proposed dimensions. I retained the original problem statement and five business questions since the completed warehouse supports all of them with the available records. I also kept the operational database unchanged. Its 16 tables, 30 check constraints, and v_trade_pnl view from Milestone 2 contain the information required for the ETL process. I did make several refinements to the candidate model presented in Section 8 of Milestone 2:

- **Economic events:** the separate Economic Event dimension was folded into the Date dimension as a second hierarchy (Date, MajorEvent, EventDayType). The source calendar never has more than one high-impact release on the same day, so each date has at most one major event and no bridge table is needed.
- **Trading sessions: the Session dimension became a Time-of-Day dimension at minute grain, with Session and Hour as two separate parent levels. This preserves the precise entry time while still allowing analysis by session.**
- **Platform:** Platform is an attribute of the Account level instead of a separate dimension, because it has only two values and never changes for an account.
- **Rule violations: violations are stored as measures of the Daily Balance snapshot rather than in a fifth fact. Every one of the 143 violations falls on an account and day that already has a balance row, meaning both records share the same grain.**
- **Cash flow: fees and payouts were combined into one Cash Flow fact with a signed amount, allowing net return to be calculated by summing across all fees and payouts.**

With these changes, both the operational database and the data warehouse are final for this project. The next milestone will change only how the warehouse is loaded, by

rebuilding the SQL load as a Talend job with the incremental and Type 2 logic described in Section 6.2, while the tables, keys, and constraints stay as documented here.

3. Primary Events

A primary event represents a business activity that the warehouse must record in the warehouse. Each event determines the level of detail, or grain, of its corresponding fact table. T 1 identifies the main TradeOps events, their operational sources, and the facts used to represent them.

Primary event	Description	Source (tradeops)	Fact (grain)
Trade closed	A leader account opens and closes a position in one contract month	Trade, Fill, v_trade_pnl	Trade (one row per trade)
Trade copied	The copier repeats a leader trade in a follower account, possibly in the micro contract	Trade.LeaderTradeID	Trade (role = Copy)
End-of-day statement	The firm reports start and end balance, high-water mark, and drawdown floor	DailyAccountBalance	Daily Balance (account x day)
Rule violated	A drawdown, daily loss, or consistency rule is breached	RuleViolation	Daily Balance (violation counts)
Fee charged	Evaluation, activation, reset, or monthly fee is billed	AccountCharge	Cash Flow (one row per charge)
Payout paid or denied	A funded account requests a withdrawal and receives the trader share	Payout	Cash Flow (one row per request)
Account opened, passed, failed, or breached	An account moves through its lifecycle	Account	Account Lifecycle (one row per account)

T 1. Primary events and the facts that record them

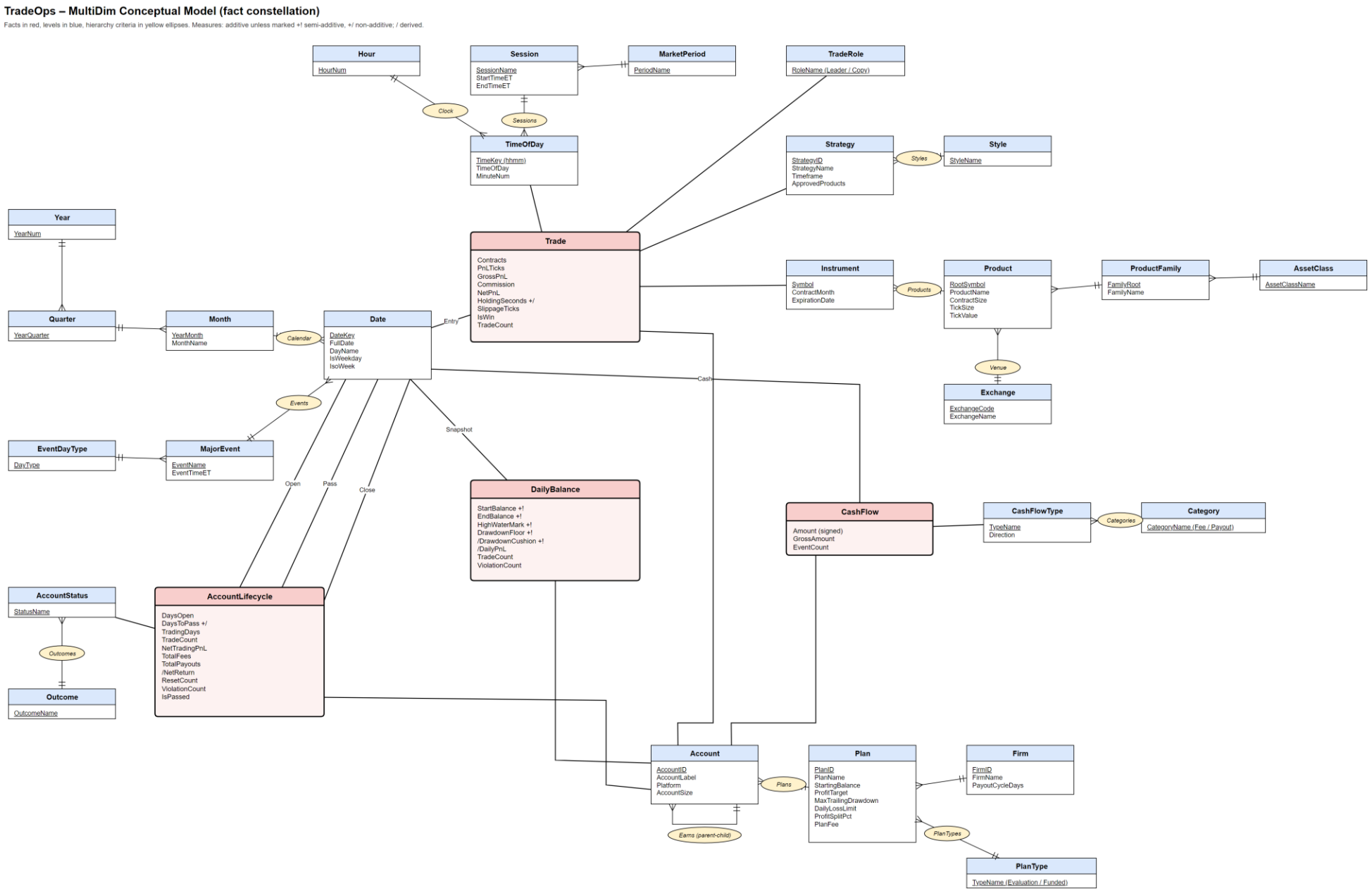
4. Conceptual Model (MultiDim)

I developed the conceptual diagram in draw.io using the MultiDim notation covered in Chapter 4 of Vaisman and Zimányi (2022). The model is presented in F 1, and its editable version is saved as TradeOps_DW_Diagrams.drawio. Red rectangles indicate facts and their measures, whereas blue rectangles display dimension levels with underlined identifiers. One-to-many relationships connect the levels within each hierarchy, and yellow ellipses identify the hierarchy criteria. In this notation, measures are additive by default; +! indicates semi-additive measures, +/ indicates non-additive measures, and a leading slash identifies a derived measure. Since the complete model is extensive, F 4 to F 7 in Appendix A show its four quadrants enlarged.

The proposed design follows a fact constellation, with four fact tables connected through shared Date and Account dimensions. Using these conformed dimensions makes it

possible for measures from different facts to be compared using drill-across analysis. The four facts are:

- **Trade: a transaction fact with one row per closed trade (6,322 rows). It connects to Date, Time of Day, Account, Instrument, Strategy, and Trade Role, and carries P&L, commission, contracts, holding time, and copy slippage.**
- **Daily Balance:** a periodic snapshot with one row per account per trading day (2,916 rows). Its balances are level measures, so they can be summed across accounts but not across days.
- **Cash Flow: a transaction fact with one row per fee charge or payout request (283 rows). Fee transactions are recorded as negative values and payouts received as positive amounts.**
- **Account Lifecycle:** an accumulating snapshot with one row per account (134 rows). It links to Date three times, in the roles Open, Pass, and Close, and records how long the account lasted and what it earned.



F 1. TradeOps MultiDim conceptual model (fact constellation)

4.1 Dimensions and Hierarchies

T 2 outlines each dimension, its hierarchy levels, and the type of hierarchy used. Several use the specialized hierarchy structures discussed in Chapter 4 of Vaisman and Zimányi (2022), as explained below.

Dimension	Hierarchies (leaf to root)	Hierarchy type	Used by
Date	Calendar: Date -> Month -> Quarter -> Year; Events: Date -> MajorEvent -> EventDayType	Two parallel hierarchies; role-playing (Open, Pass, Close)	All four facts
TimeOfDay	Sessions: Minute -> Session -> MarketPeriod; Clock: Minute -> Hour	Parallel hierarchies sharing the leaf	Trade
Account	Plans: Account -> Plan -> Firm; PlanTypes: Plan -> PlanType; Earns: Account -> Account	Balanced, plus a recursive (parent-child) hierarchy	All four facts
Instrument	Products: Instrument -> Product -> ProductFamily -> AssetClass; Venue: Product -> Exchange	Balanced, with a branch at Product	Trade
Strategy	Styles: Strategy -> Style	Balanced	Trade
TradeRole	Leader / Copy (single level)	-	Trade
CashFlowType	Categories: CashFlowType -> Category	Balanced	Cash Flow
AccountStatus	Outcomes: Status -> Outcome	Balanced	Account Lifecycle

T 2. Dimensions and hierarchies

- **Parallel hierarchies:** Date has two independent hierarchies. The Calendar hierarchy groups days into months and quarters, while the Events hierarchy groups them by the major release of the day (CPI, Nonfarm Payrolls, FOMC, or None). Time of Day is handled the same way, with Session and Hour as separate parents.
- **Recursive hierarchy:** an evaluation account earns a funded account, so Account is related to itself. This parent-child hierarchy is used to follow one trader's path from the evaluation through the funded account.
- **Role-playing dimension:** Account Lifecycle uses Date three times with different meanings (Open, Pass, Close). The diagram shows the roles as labels on the three links.
- **Product family: the ProductFamily level groups a micro contract with its full-size product (MNQ with NQ), using the FullSizeRoot relationship added in Milestone 2. This makes it possible to compare a copied micro trade with a leader trade in the full-size contract.**

4.2 Measures and Additivity

As explained in Section 3.1.2 of Vaisman and Zimányi (2022), the appropriate aggregation method depends on the kind of value it represents. T 3 identifies the measures in the model, their additivity, and the aggregation methods used in the OLAP queries.

Fact	Measure	Type	Additivity	Valid aggregation
Trade	NetPnL, GrossPnL, Commission, Contracts, PnLTicks, TradeCount	Flow	Additive	SUM
Trade	SlippageTicks (copies only)	Flow	Additive	SUM, AVG
Trade	HoldingSeconds	Unit	Non-additive (+/)	AVG, MIN, MAX
Trade	IsWin	Flag	Additive count	SUM, then divide by TradeCount for win rate
Daily Balance	StartBalance, EndBalance, HighWaterMark, DrawdownFloor, DrawdownCushion	Level	Semi-additive (+!)	SUM across accounts; AVG or last value over time
Daily Balance	DailyPnL, TradeCount, ViolationCount	Flow	Additive	SUM
Cash Flow	Amount (signed), GrossAmount, EventCount	Flow	Additive	SUM
Account Lifecycle	NetTradingPnL, TotalFees, TotalPayouts, NetReturn, ResetCount, ViolationCount	Flow	Additive	SUM
Account Lifecycle	DaysOpen, TradingDays	Flow	Additive	SUM, AVG
Account Lifecycle	DaysToPass	Unit	Non-additive (+/)	AVG, MIN, MAX
Account Lifecycle	IsPassed	Flag	Additive count	SUM, then divide by count for pass rate

T 3. Measures with their type and valid aggregation

Win and pass rates are computed instead of stored since adding percentages across groups would be misleading across groups. The warehouse instead stores the underlying counts, such as IsWin and TradeCount or IsPassed and the total number of evaluations. Each query uses these counts to calculate the appropriate percentage for the level being analyzed.

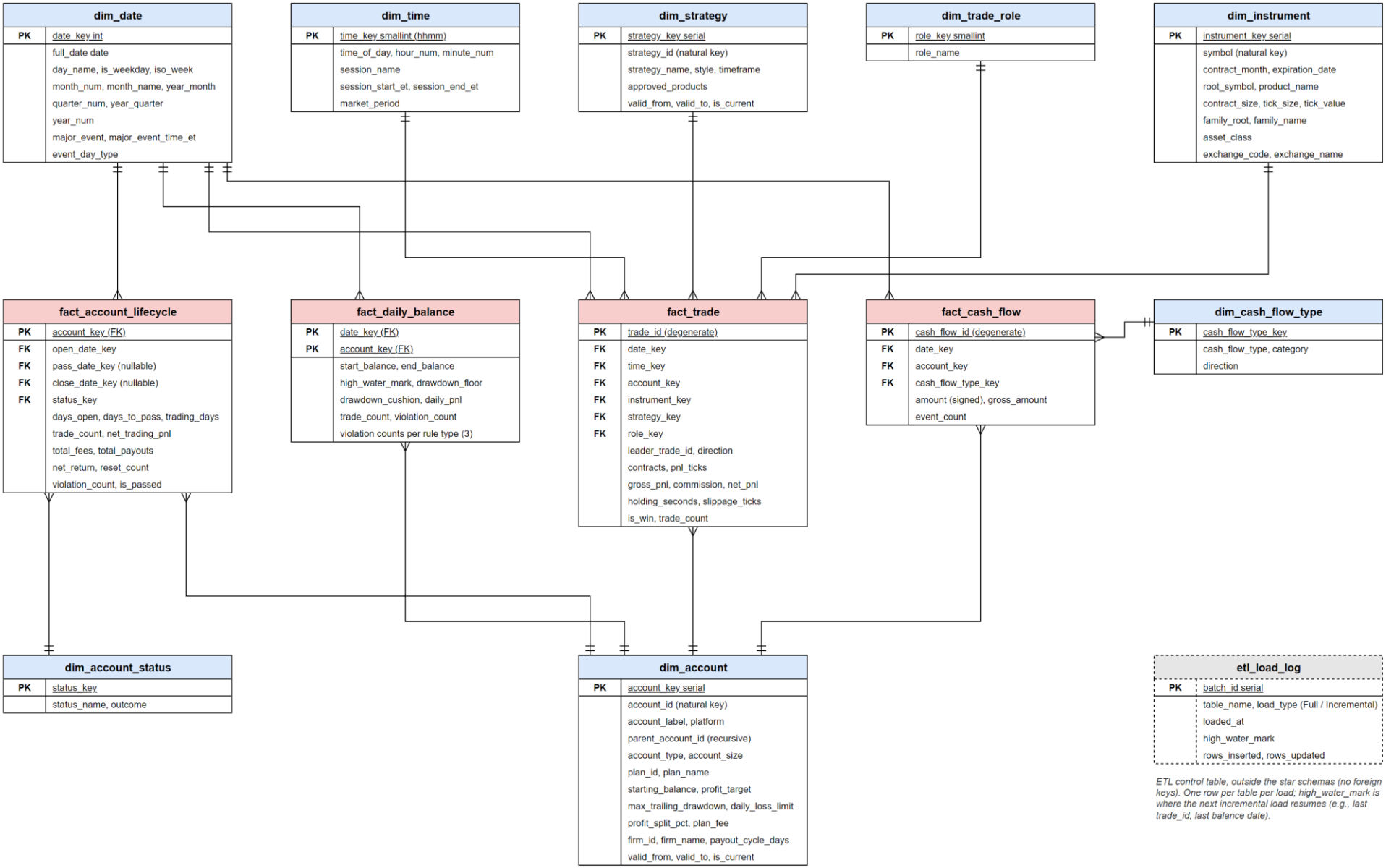
5. Logical Model (ROLAP)

I converted the conceptual design into a relational schema using the mapping guidelines in Chapter 5 of Vaisman and Zimányi (2022). The resulting structure is a constellation of star schemas, where each dimension is represented by one denormalized table and the four facts reuse common dimensions. I evaluated a snowflake design as well to reduce

repeated plan and firm information in dim_account. Because the largest dimension has only 1,440 rows, so the reduction in storage would be small, and the additional tables would introduce additional joins in OLAP queries. F 2 presents the chosen schema. F 8 to F 11 in Appendix A show the four quadrants of the schema enlarged.

TradeOps – ROLAP Schema in PostgreSQL (schema tradeops_dw: 4 fact tables, 8 dimension tables, 1 ETL control table)

Fact constellation of star schemas: every dimension is one denormalized table shared by the facts that use it. Fact tables in red, dimension tables in blue, ETL control table in grey.



F 2. ROLAP schema in PostgreSQL (schema tradeops_dw)

5.1 Mapping Rules

Conceptual element	Relational mapping	Result in tradeops_dw
Dimension with a hierarchy	One table holding all levels; each level becomes a set of columns	dim_account holds Account, Plan, Firm, and PlanType columns
Level identifier	New surrogate key as primary key; source key kept as an attribute	account_key (serial) with account_id; instrument_key with symbol
Fact relationship	One table; a foreign key to each dimension and one column per measure	fact_trade with six foreign keys and its measures
Role-playing dimension	One foreign key per role to the same table	fact_account_lifecycle: open_date_key, pass_date_key, close_date_key
Recursive hierarchy	Parent identifier stored in the child row	dim_account.parent_account_id
Time dimension	Date table generated for the full period, with a smart integer key	dim_date: 730 days, date_key = yyyymmdd
Degenerate dimension	Transaction number kept in the fact table without a dimension table	trade_id, leader_trade_id, direction, cash_flow_id
Slowly changing dimension	Type 2 columns: valid_from, valid_to, is_current; facts look up the version valid on the event date	dim_account and dim_strategy

T 4. Mapping of the conceptual model to the relational model

Consistent with the Milestone 2 recommendation, two dimensions support Type 2 slowly changing dimensions. If plan fees or rules are revised, or when a strategy's permitted products are updated, ETL can end the current record by setting valid_to and create a replacement with a new surrogate key. Fact records are connected to whichever version was active on the event date, maintaining the historical rules that applied at that time. Because the current dataset has no such updates, each account and strategy currently has just one version. The Account Lifecycle fact is handled differently because it is an accumulating snapshot with one row per account. Its row always points to the current version of the account, so when a new version is created, the load re-points the existing row to the new surrogate key instead of adding a second row, which keeps the grain at one row per account.

5.2 Tables

Table	Type	Rows	Primary key	Description
dim_date	Dimension	730	date_key	Every day of 2025-2026 with calendar levels and the major event
dim_time	Dimension	1,440	time_key	Every minute of the day with its session and market period
dim_account	Dimension	134	account_key	Account, plan, firm, and plan type; SCD Type 2
dim_instrument	Dimension	164	instrument_key	Contract month, product, family, asset class, exchange
dim_strategy	Dimension	5	strategy_key	Strategy and style; SCD Type 2
dim_trade_role	Dimension	2	role_key	Leader or Copy
dim_cash_flow_type	Dimension	6	cash_flow_type_key	Fee types and payout outcomes

dim_account_status	Dimension	5	status_key	Final account status and outcome
fact_trade	Fact	6,322	trade_id	One row per closed trade
fact_daily_balance	Fact	2,916	(date_key, account_key)	One row per account per trading day
fact_cash_flow	Fact	283	cash_flow_id	One row per fee charge or payout request
fact_account_lifecycle	Fact	134	account_key	One row per account
etl_load_log	ETL control	9	batch_id	One row per table per load, with the high-water mark for the next incremental load

T 5. Tables in schema tradeops_dw

6. PostgreSQL Implementation

I implemented the warehouse in a separate tradeops_dw schema within the same PostgreSQL 16 database as the operational tables. This arrangement allows me to load the warehouse directly with INSERT ... SELECT statements without exporting and importing separate files. Four additional scripts extend the four scripts developed in Milestone 2:

Script	Purpose
sql/05_dw_schema.sql	Drops and recreates tradeops_dw: 8 dimension tables, 4 fact tables, the etl_load_log control table, 16 foreign keys, 26 check constraints, 2 partial unique indexes, and 8 indexes
sql/06_dw_load.sql	Initial full load in one transaction: generates the date and time dimensions, flattens the other dimensions, loads the facts with surrogate-key lookups, and logs the high-water marks
sql/07_olap_queries.sql	The nine OLAP operations in Section 7 (output in sql/07_olap_output.txt)
sql/08_dw_checks.sql	Reconciles every fact with its source, lists the load log and the constraints on each table (output in sql/08_dw_checks_output.txt)

T 6. Milestone 3 scripts

The complete database, operational and analytical, can be rebuilt from the project folder with:

```
psql -d tradeops -f sql/01_schema.sql -f sql/02_load.sql -f sql/03_views.sql
psql -d tradeops -f sql/05_dw_schema.sql -f sql/06_dw_load.sql
psql -d tradeops -f sql/07_olap_queries.sql -f sql/08_dw_checks.sql
```

6.1 ETL Process

The ETL process populates dimension tables before fact tables, following the standard dimensional-modeling sequence. This sequence ensures that the required surrogate keys exist when the fact rows are inserted. The process consists of the following steps:

- **Generated dimensions:** dim_date is generated with generate_series for every day of 2025 and 2026 and is joined to the economic calendar to add the major event of each day. dim_time is generated for all 1,440 minutes and joined to the trading sessions,

including the Asia session that wraps past midnight. The hour from 17:00 to 18:00 ET, when CME is closed for maintenance, is labeled Maintenance.

- **Flattened dimensions:** dim_account joins Account, AccountPlan, and PropFirm into one row per account, and dim_instrument joins Instrument, Product, the full-size family product, and Exchange.
- **Fact transformations:** fact_trade reads the view v_trade_pnl and joins each copied trade to its leader to calculate slippage in ticks. fact_daily_balance adds same-day trade and violation counts. fact_cash_flow signs each amount, and fact_account_lifecycle aggregates each account's trades, fees, payouts, resets, and violations into a single row.
- **Surrogate-key lookup:** each fact joins dim_account and dim_strategy on the natural key and on the condition that the event date falls between valid_from and valid_to, which is the lookup an SCD Type 2 dimension needs.

6.2 Load Types and Refresh Strategy

Together, 05_dw_schema.sql and 06_dw_load.sql perform the initial full load: the first script recreates every table empty, and the second fills them in a single transaction, which is appropriate for the first load and for the current volume of about 9,650 fact rows. A daily full refresh would be inefficient in production, however, because it would reprocess history that has not changed and, more importantly, it would erase the older versions kept by the Type 2 dimensions. Therefore, each table is assigned the load type that matches how its source data changes, as summarized in T 7.

Table	Load type	Change detection	Action on change
dim_date, dim_time	Generated (static)	Calendar range or economic calendar extended	dim_date is extended with generate_series; a newly scheduled release overwrites major_event (Type 1)
dim_trade_role, dim_cash_flow_type, dim_account_status	Static reference	Code list changes (rare)	Full reload of the small table
dim_instrument	Incremental insert	Symbol not yet in dim_instrument (new contract month after a roll)	Insert the new contract; name corrections overwrite (Type 1)
dim_account	Incremental, SCD Type 2	New AccountID, or plan rule and fee columns differ from the current row	Close the current row (valid_to, is_current = false) and insert a new version with a new account_key
dim_strategy	Incremental, SCD Type 2	New StrategyID, or style, timeframe, or approved products differ	Same as dim_account; a rename overwrites strategy_name (Type 1)
fact_trade	Incremental append	trade_id above the high-water mark	Insert only; a closed trade never changes

fact_daily_balance	Incremental append	Trading date after the high-water mark	Insert the new account-days with their trade and violation counts
fact_cash_flow	Incremental append by key	Charge or payout ID not yet in the fact; payouts only once Paid or Denied	Insert; a payout still Requested is picked up by a later run once it is resolved
fact_account_lifecycle	Accumulating snapshot update	Account with new trades, charges, payouts, or a status change	Recompute and update the row in place; fill pass_date_key and close_date_key when reached

T 7. Load type and change handling for each warehouse table

To support incremental processing, the schema includes a control table, etl_load_log, that sits outside the star schemas. Every load writes one row per table with the load type, the number of rows inserted and updated, and a high-water mark that tells the next run where to resume. The initial full load recorded a high-water mark of trade_id 6,322 for fact_trade and of 2026-08-31 for fact_daily_balance, so the next run only needs to read source rows beyond those values. The attributes of the two slowly changing dimensions are also split by how their changes should be treated:

- **Type 2 (history kept):** the plan rule and fee columns of dim_account (profit_target, max_trailing_drawdown, daily_loss_limit, profit_split_pct, plan_fee, and payout_cycle_days) and the style, timeframe, and approved_products of dim_strategy. These values decide whether a trade broke a rule, so older facts must stay linked to the values that were in force when they happened.
- **Type 1 (overwrite):** account_label, platform, strategy_name, and the product and exchange names. A change to these is usually a correction or a rename that does not change the meaning of any fact, so keeping the old value would only add duplicate rows.

6.3 Integrity Constraints

Some warehouses omit constraints to speed up loading, but the overhead is minimal at this scale, and they catch ETL errors when the data is loaded instead of when a wrong number appears in a report. In addition to 13 primary keys, 16 foreign keys, and 7 unique keys, the schema declares 26 CHECK constraints and 2 partial unique indexes, which fall into five groups:

- **Domain rules:** the flags is_win and is_passed must be 0 or 1, contract_size must be Micro or Full-size, contracts must be positive, and commissions, holding times, and counts cannot be negative.
- **Calculated measures:** every stored calculated measure must equal its formula, for example net_pnl = gross_pnl - commission, drawdown_cushion = end_balance - drawdown_floor, net_return = total_payouts - total_fees, and violation_count = the sum

of the three violation types. As a result, a calculated measure can never disagree with the values it is derived from.

- **Smart keys:** date_key must equal full_date written as yyyymmdd, and time_key must equal the hour and minute written as hhmm, so the smart keys cannot drift from the attributes they encode.
- **SCD Type 2 rules:** valid_from cannot be later than valid_to, and a partial unique index on account_id (and on strategy_id) WHERE is_current allows only one current version of each account and strategy, which is the condition that the surrogate-key lookup and the Account Lifecycle fact depend on.
- **Role consistency:** a copied trade must have both a leader_trade_id and a slippage value, while a leader trade must have neither, and an account has a pass date only if it also has a days_to_pass value.

Every record in the initial load meets every constraint, and sql/08_dw_checks.sql now lists the constraints declared on each table together with the load log.

6.4 Reconciliation

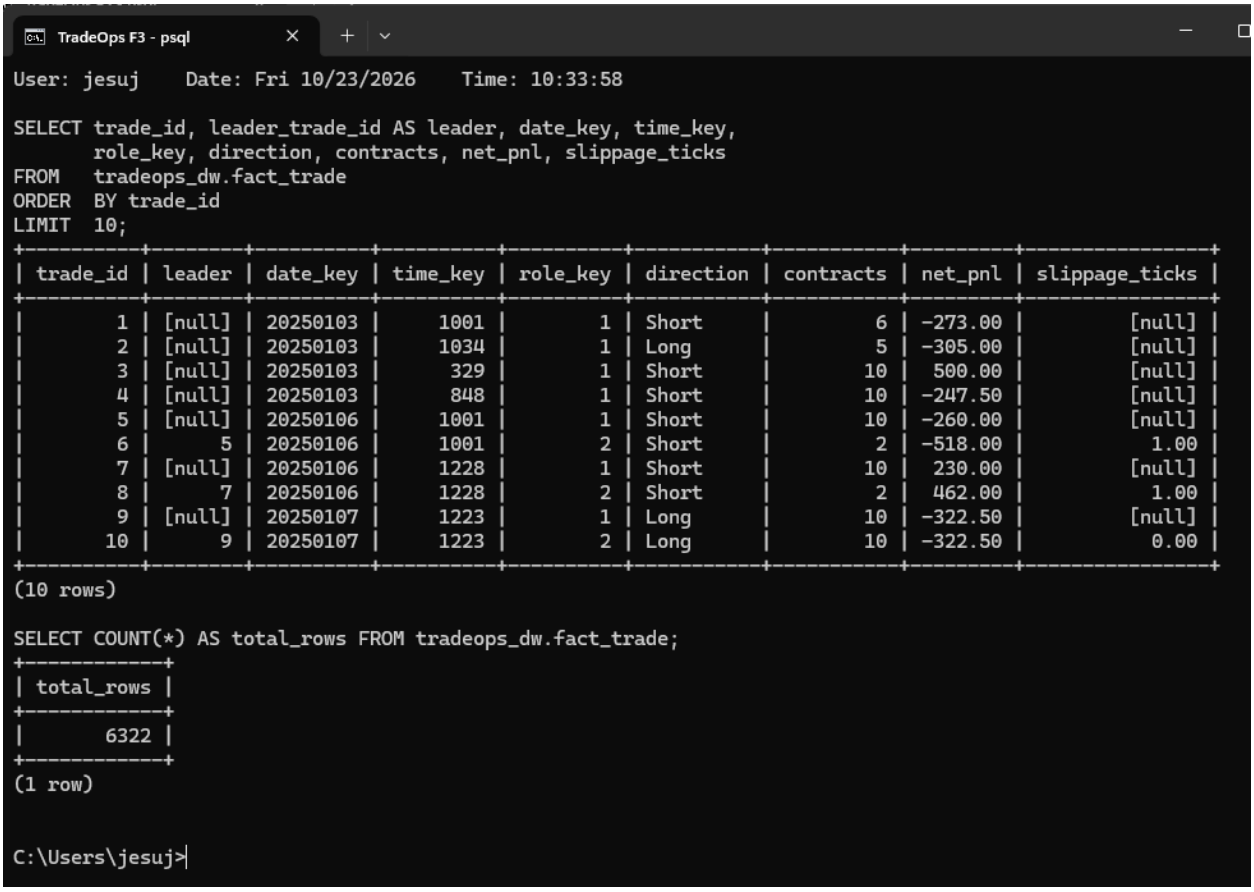
The reconciliation script, sql/08_dw_checks.sql, compares warehouse totals and record counts against the original operational tables. All reconciliation checks were successful:

Check	Warehouse	Source	Result
Trade rows	6,322	6,322	Match
Daily balance rows	2,916	2,916	Match
Cash flow rows	283	220 charges + 63 payouts	Match
Lifecycle rows	134	134 accounts	Match
Total net P&L	$15,381.50	$15,381.50	Match
Total fees	$17,497.60	$17,497.60	Match
Total payouts received	$58,950.00	$58,950.00	Match
Rule violations	143	143	Match
Net return, Lifecycle fact vs. Cash Flow fact	$41,452.40	$41,452.40	Consistent

T 8. Warehouse reconciliation results

F 3 shows the first ten rows of fact_trade, queried with psql on the local PostgreSQL 16 server after the load, followed by the total row count of 6,322. The query lists the surrogate keys, the degenerate attributes leader_trade_id and direction, and the main measures, which are the columns needed to follow the copy-trading example. Trade 6 copies leader trade 5, as shown previously in Milestone 2, and its slippage_ticks value of 1.00 indicates

that the copy closed one tick worse than the original trade. Leader trades have NULL slippage values because there is no earlier trade to compare them against.



F 3. First rows of tradeops_dw.fact_trade and its total row count, queried with psql on the local PostgreSQL 16 server

7. OLAP Operations

To approach the project from the business owner's perspective, I selected analyses that would be useful for reviewing operations each month. Each analysis applies an OLAP operation from Section 3.2 of Vaisman and Zimányi (2022) and is implemented with SQL against the ROLAP warehouse. T 9 gives an overview of the nine operations, followed by their results and business implications. To keep the section readable, each SQL listing shows only the clauses that define the operation, while the complete queries for all nine operations are in sql/07_olap_queries.sql and their full output is in sql/07_olap_output.txt.

#	Operation	Fact	Dimensions and levels	Question
OP1	Roll-up	Cash Flow	Account: Plan -> Firm -> All	BQ1
OP2	Drill-down	Trade	Date: Year -> Quarter -> Month	BQ1
OP3	Slice	Trade	Date.EventDayType = Major event day; by MajorEvent and Strategy	BQ3

OP4	Dice	Trade	AssetClass, Session, Year, and TradeRole restricted together	BQ2, BQ3
OP5	Pivot	Trade	Strategy (rows) × AssetClass (columns)	BQ2
OP6	Drill-across	Trade + Cash Flow	Firm × Quarter (conformed Account and Date)	BQ1
OP7	Roll-up with CUBE	Trade	ProductFamily × ContractSize, copies only	BQ5
OP8	Drill-across + rank	Daily Balance + Trade	Strategy on account-days with violations	BQ2
OP9	Roll-up of a non-additive measure	Account Lifecycle	Account: Plan -> Firm, evaluations only	BQ4

T 9. Planned OLAP operations

OP1. Roll-up: net return by plan and firm

This query aggregates the Cash Flow fact through the Account hierarchy, moving from individual plans to firms and then to the overall total. GROUP BY ROLLUP returns these three levels of aggregation together.

```
SELECT a.firm_name, a.plan_name,
    SUM(f.amount) FILTER (WHERE t.category = 'Fee')    AS fees,
    SUM(f.amount) FILTER (WHERE t.category = 'Payout') AS payouts,
    SUM(f.amount) AS net_return
FROM fact_cash_flow f JOIN dim_account a USING (account_key)
JOIN dim_cash_flow_type t USING (cash_flow_type_key)
GROUP BY ROLLUP (a.firm_name, a.plan_name);
```

firm	plan	fees	payouts	net_return
Lucid Trading	LucidFlex 50K Eval	-3,312.00		-3,312.00
Lucid Trading	LucidFlex 50K Funded		7,290.00	7,290.00
Lucid Trading	* All plans	-3,312.00	7,290.00	3,978.00
MyFundedFutures	Rapid 50K Eval	-4,943.40		-4,943.40
MyFundedFutures	Rapid 50K Funded	-1,419.00	11,250.00	9,831.00
MyFundedFutures	* All plans	-6,362.40	11,250.00	4,887.60
Topstep	150K Combine	-2,652.20		-2,652.20
Topstep	150K Funded	-447.00	16,560.00	16,113.00
Topstep	50K Combine	-3,234.00		-3,234.00
Topstep	50K Funded	-1,490.00	23,850.00	22,360.00
Topstep	* All plans	-7,823.20	40,410.00	32,586.80
*** All firms**	* All plans	-17,497.60	58,950.00	41,452.40

T 10. OP1 result: net return rolled up from plan to firm

After accounting for all fees, the business generated $41,452.40 in net return. Evaluation plans show negative returns individually because they incur costs without producing payouts. Funded accounts, however, generated enough income to cover those costs. Topstep contributed the most at $32,586.80, with much of that amount coming from its 50K Funded plan.

OP2. Drill-down: net P&L from year to month

The analysis first groups the Trade fact by year. It then moves down the Calendar hierarchy to quarterly results and finally to the individual months within a selected quarter.

```
SELECT d.year_num, SUM(f.net_pnl) AS net_pnl, SUM(f.trade_count) AS trades
FROM fact_trade f JOIN dim_date d USING (date_key)
GROUP BY d.year_num;
-- drill down: GROUP BY d.year_quarter, then GROUP BY d.year_month
-- with WHERE d.year_quarter = '2026-Q2'
```

year_num	net_pnl	trades
2025	-64,999.50	3386
2026	80,381.00	2936

T 11. OP2 result at the Year level

year_quarter	net_pnl	trades
2025-Q1	-7,739.50	745
2025-Q2	-16,473.00	1026
2025-Q3	-13,193.00	775
2025-Q4	-27,594.00	840
2026-Q1	-14,919.50	746
2026-Q2	65,437.50	1056
2026-Q3	29,863.00	1134

T 12. OP2 result at the Quarter level

year_month	net_pnl	trades
2026-04	-15,941.00	339
2026-05	18,037.00	299
2026-06	63,341.50	418

T 13. OP2 result at the Month level for 2026-Q2

The yearly totals show a loss of $64,999.50 in 2025 and a gain of $80,381.00 in 2026. The quarterly view reveals losses in every quarter through 2026-Q1, followed by a positive shift in 2026-Q2. At the monthly level, June 2026 contributed $63,341.50, indicating that a large share of the improvement came from one month and may not represent consistent performance.

OP3. Slice: performance on major event days

This slice filters the Date dimension's Events hierarchy to include only CPI, Nonfarm Payrolls, and FOMC release days. The remaining trades are then summarized by economic event and trading strategy.

```
SELECT d.major_event, s.strategy_name, SUM(f.net_pnl) AS net_pnl,
    ROUND(100.0 * SUM(f.is_win) / SUM(f.trade_count), 1) AS win_rate_pct
FROM fact_trade f JOIN dim_date d USING (date_key) JOIN dim_strategy s USING (strategy_key)
WHERE d.event_day_type = 'Major event day'
GROUP BY d.major_event, s.strategy_name;
```

major_event	strategy_name	trades	net_pnl	win_rate_pct
CPI	London Breakout	58	1,974.00	41.4
CPI	VWAP Reversion	72	-1,773.50	47.2
CPI	Opening Range Breakout	69	-7,328.50	33.3
CPI	News Fade	103	-8,654.50	37.9
CPI	Trend Pullback	77	-15,809.00	15.6
FOMC Rate Decision	Opening Range Breakout	51	2,074.00	52.9
FOMC Rate Decision	Trend Pullback	54	-4,106.00	20.4
FOMC Rate Decision	London Breakout	40	-6,600.00	17.5
FOMC Rate Decision	VWAP Reversion	38	-9,812.00	15.8
Nonfarm Payrolls	News Fade	112	10,739.00	65.2
Nonfarm Payrolls	Trend Pullback	72	7,710.00	45.8
Nonfarm Payrolls	VWAP Reversion	97	5,034.00	69.1
Nonfarm Payrolls	London Breakout	68	897.00	42.6
Nonfarm Payrolls	Opening Range Breakout	55	-11,080.00	20.0

T 14. OP3 result: trades on major event days by event and strategy

Performance appears to vary substantially by economic event. Four of the five strategies were profitable on Nonfarm Payrolls days, including News Fade, which earned $10,739.00 and achieved a 65.2% win rate. By comparison, most strategies lost money on CPI and FOMC days. Trend Pullback performed especially poorly on CPI days, winning only 15.6% of its trades. These findings suggest that the business should consider avoiding trend-based strategies during CPI and FOMC releases.

OP4. Dice: equity-index leader trades in the US morning of 2026

A dice operation narrows the data to a specific combination of dimension values. Here, the selected trades are from the Equity Index asset class, occurred during NY Open or NY Midday in 2026, and belong to leader accounts.

```
SELECT i.family_name, t.session_name, SUM(f.trade_count) AS trades,
    SUM(f.net_pnl) AS net_pnl,
    ROUND(AVG(f.holding_seconds) / 60.0, 1) AS avg_hold_min
FROM fact_trade f JOIN dim_instrument i USING (instrument_key)
JOIN dim_time t USING (time_key) JOIN dim_date d USING (date_key)
JOIN dim_trade_role r USING (role_key)
WHERE i.asset_class = 'Equity Index' AND t.session_name IN ('NY Open', 'NY Midday')
 AND d.year_num = 2026 AND r.role_name = 'Leader'
GROUP BY i.family_name, t.session_name;
```

family_name	session_name	trades	net_pnl	avg_hold_min
E-mini Dow	NY Open	32	2,055.00	48.8
E-mini Nasdaq-100	NY Open	66	2,187.00	55.8
E-mini Russell 2000	NY Midday	48	2,012.00	14.7
E-mini S&P 500	NY Midday	39	-75.50	14.8
E-mini S&P 500	NY Open	37	-2,107.50	50.8

T 15. OP4 result: the diced sub-cube by product family and session

Within these filters, Nasdaq-100, Dow, and Russell 2000 trades produced positive results, while S&P 500 trades were unprofitable in both sessions, particularly during NY Open. Holding times also differed: NY Midday positions lasted about 15 minutes on average, compared with roughly 50 minutes during the opening session.

OP5. Pivot: strategy by asset class

The pivot rearranges the results so asset classes appear as columns instead of rows. This makes it easier to compare how each strategy performs across the three market categories.

```
SELECT s.strategy_name,
 SUM(f.net_pnl) FILTER (WHERE i.asset_class = 'Equity Index') AS equity_index,
 SUM(f.net_pnl) FILTER (WHERE i.asset_class = 'Energy')      AS energy,
 SUM(f.net_pnl) FILTER (WHERE i.asset_class = 'Metals')      AS metals
FROM fact_trade f JOIN dim_strategy s USING (strategy_key)
JOIN dim_instrument i USING (instrument_key) GROUP BY s.strategy_name;
```

strategy_name	equity_index	energy	metals	total
Trend Pullback	3,949.00	17,662.00	14,064.00	35,675.00
London Breakout		67,186.00	-45,152.00	22,034.00
News Fade	2,084.50			2,084.50
Opening Range Breakout	-10,898.50			-10,898.50

VWAP Reversion	-33,513.50			-33,513.50

T 16. OP5 result: net P&L pivoted by asset class

The findings indicate that London Breakout's overall profit depends on its strong energy performance of $67,186.00, which offsets a $45,152.00 loss in metals. Excluding gold from that strategy's approved instruments would have produced the largest improvement. VWAP Reversion also lost money in its only asset class, making it a possible strategy to discontinue.

OP6. Drill-across: trading profit versus cash flow

A drill-across analysis compares measures from separate fact tables by using dimensions they share. In this example, the Trade and Cash Flow facts are each summarized by firm and quarter, and their aggregated results are combined at those common levels.

```
WITH trading AS (SELECT firm_name, year_quarter, SUM(net_pnl) AS trading_pnl ...),
   cash    AS (SELECT firm_name, year_quarter, fees_paid, payouts_received ...)
SELECT ... FROM trading FULL JOIN cash USING (firm_name, year_quarter);
```

firm	quarter	trading_pnl	fees_paid	payouts_received	net_return
Lucid Trading	2025-Q1	-2,738.00	368.00	0.00	-368.00
Lucid Trading	2025-Q2	-5,583.00	768.00	540.00	-228.00
Lucid Trading	2025-Q3	-5,544.00	736.00	0.00	-736.00
Lucid Trading	2025-Q4	-3,872.00	560.00	720.00	160.00
Lucid Trading	2026-Q1	-1,261.00	240.00	0.00	-240.00
Lucid Trading	2026-Q2	5,933.00	320.00	0.00	-320.00
Lucid Trading	2026-Q3	8,635.50	320.00	6,030.00	5,710.00
MyFundedFutures	2025-Q1	-222.00	600.60	0.00	-600.60
MyFundedFutures	2025-Q2	-553.00	1,264.80	4,410.00	3,145.20
MyFundedFutures	2025-Q3	-6,631.00	954.80	0.00	-954.80
MyFundedFutures	2025-Q4	-7,081.50	1,135.80	0.00	-1,135.80
MyFundedFutures	2026-Q1	-4,164.50	585.20	0.00	-585.20
MyFundedFutures	2026-Q2	9,708.00	627.60	1,800.00	1,172.40
MyFundedFutures	2026-Q3	6,635.00	1,193.60	5,040.00	3,846.40
Topstep	2025-Q1	-4,779.50	1,237.00	0.00	-1,237.00
Topstep	2025-Q2	-10,337.00	1,124.80	2,610.00	1,485.20
Topstep	2025-Q3	-1,018.00	245.00	0.00	-245.00
Topstep	2025-Q4	-16,640.50	1,364.40	720.00	-644.40
Topstep	2026-Q1	-9,494.00	695.80	0.00	-695.80
Topstep	2026-Q2	49,796.50	2,564.20	5,670.00	3,105.80
Topstep	2026-Q3	14,592.50	592.00	31,410.00	30,818.00

T 17. OP6 result: Trade and Cash Flow facts by firm and quarter

These findings illustrate the delay between earning trading profits and receiving cash. Topstep generated $49,796.50 in trading P&L during 2026-Q2, while a large portion of the related payouts, totaling $31,410.00, was received in 2026-Q3 after the payout process and profit split. For planning purposes, cash receipts may therefore follow trading performance by approximately one quarter.

OP7. Roll-up with CUBE: copy slippage by product family and contract size

This analysis groups copied trades through the Instrument hierarchy. CUBE calculates results for every combination of product family and contract size, along with subtotals by family, by size, and across all copies. Average slippage is reported in ticks, while its financial impact is totaled in dollars.

family	contract_size	copies	avg_slippage_ticks	slippage_cost_usd
Crude Oil	Full-size	138	0.49	1,340.00
Crude Oil	Micro	1001	0.61	6,140.00
Crude Oil	* All sizes	1139	0.60	7,480.00
E-mini Dow	Full-size	51	0.69	350.00
E-mini Dow	Micro	323	0.63	1,020.00
E-mini Dow	* All sizes	374	0.64	1,370.00
E-mini Nasdaq-100	Full-size	106	0.68	720.00
E-mini Nasdaq-100	Micro	905	0.60	1,623.00
E-mini Nasdaq-100	* All sizes	1011	0.61	2,343.00
E-mini Russell 2000	Full-size	65	0.58	380.00
E-mini Russell 2000	Micro	543	0.58	1,585.00
E-mini Russell 2000	* All sizes	608	0.58	1,965.00
E-mini S&P 500	Full-size	113	0.73	2,050.00
E-mini S&P 500	Micro	997	0.59	7,325.00
E-mini S&P 500	* All sizes	1110	0.60	9,375.00
Gold	Full-size	121	0.56	674.00
Gold	Micro	945	0.61	2,877.00
Gold	* All sizes	1066	0.60	3,551.00
*** All families**	Full-size	594	0.61	5,514.00
*** All families**	Micro	4714	0.60	20,570.00
*** All families**	* All sizes	5308	0.60	26,084.00

T 18. OP7 result: copy slippage by family and contract size

Among the 5,308 copied trades, the average execution difference was 0.60 ticks worse than the leader, creating $26,084.00 in total slippage costs. Micro contracts made up about 89% of the copies and contributed $20,570.00 of that amount. S&P 500 and crude oil had the highest copying costs, suggesting that improvements to copier speed would be especially valuable for those products.

OP8. Drill-across and rank: strategies on violation days

This drill-across brings together violation information from the Daily Balance fact with trades recorded at the same account-and-day level. Strategies are then ranked according to how many account-days with violations included trades from each strategy.

rank	strategy_name	violation_days	pnl_on_those_days
1	Trend Pullback	107	-33,703.00
2	London Breakout	82	-20,596.00
3	Opening Range Breakout	81	-28,727.50

4	VWAP Reversion	79	-19,514.00
5	News Fade	20	-2,769.00

T 19. OP8 result: strategies ranked by violation days

Trend Pullback traded on 107 account-days with violations and recorded a combined loss of $33,703.00 on those days. It therefore had the strongest association with accounts experiencing rule violations. News Fade appeared on only 20 such days, partly because it operates only around economic events. Alongside the findings from OP3, this supports considering smaller Trend Pullback positions in evaluation accounts.

OP9. Roll-up of a non-additive measure: pass rate and days to pass

This query examines evaluation accounts in the Account Lifecycle fact and aggregates the results from plans to firms. Because pass rates and time-to-pass values are non-additive, the analysis recalculates the rates and averages at each reporting level instead of summing them.

firm	plan	evaluations	passed	pass_rate_pct	avg_days_to_pass	avg_resets
Lucid Trading	LucidFlex 50K Eval	22	8	36.4	32.5	0.36
Lucid Trading	* All plans	22	8	36.4	32.5	0.36
MyFundedFutures	Rapid 50K Eval	33	11	33.3	39.7	0.42
MyFundedFutures	* All plans	33	11	33.3	39.7	0.42
Topstep	150K Combine	10	3	30.0	44.7	0.10
Topstep	50K Combine	37	10	27.0	38.7	0.27
Topstep	* All plans	47	13	27.7	40.1	0.23
*** All firms**	* All plans	102	32	31.4	38.1	0.32

T 20. OP9 result: evaluation pass rate rolled up from plan to firm

In total, 31.4% of evaluations were passed, and successful evaluations required an average of 38.1 days. Lucid Trading had the strongest pass rate at 36.4% and the shortest average time to pass at 32.5 days. Topstep's pass rate was the lowest at 27.7%, but OP1 shows that its funded accounts produced larger payouts, helping offset the lower success rate.

The SQL file also contains a semi-additive example, labeled OP9b in sql/07_olap_queries.sql. It first totals funded balances across accounts for each day, then averages those daily totals within each month. This avoids incorrectly counting the same balance repeatedly across multiple trading days.

8. Conclusions and Next Steps

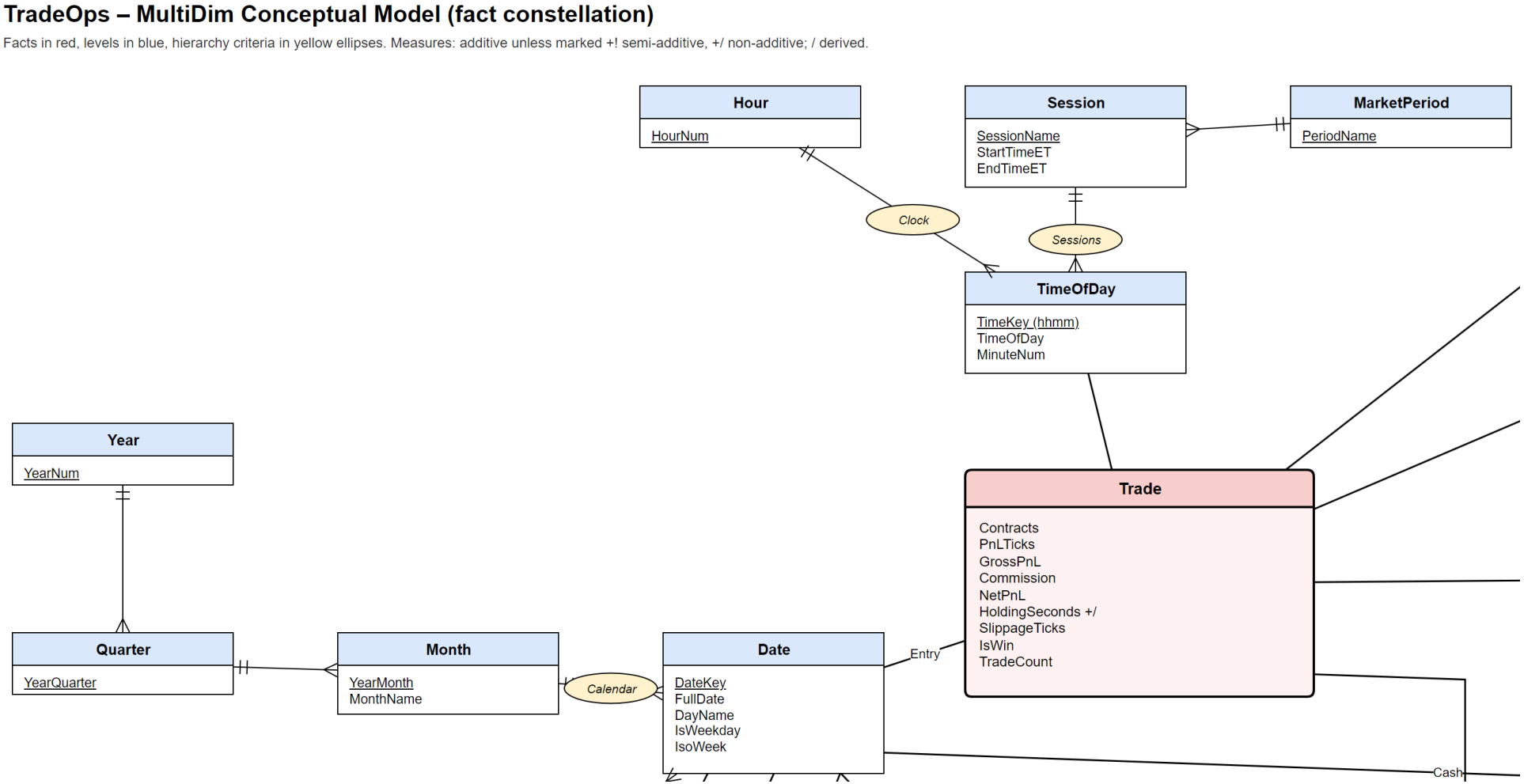
This milestone establishes the TradeOps warehouse design and its implementation as a functioning ROLAP system. The conceptual MultiDim model organizes four facts around shared Date and Account dimensions, and the PostgreSQL relational model implements them as connected star schemas. Reconciliation confirms that the

warehouse matches the operational database in both record counts and financial totals. The nine OLAP analyses address all five business questions and identify potential actions, including removing metals from London Breakout, avoiding trend strategies during CPI and FOMC releases, and reducing copying costs for S&P 500 and crude oil.

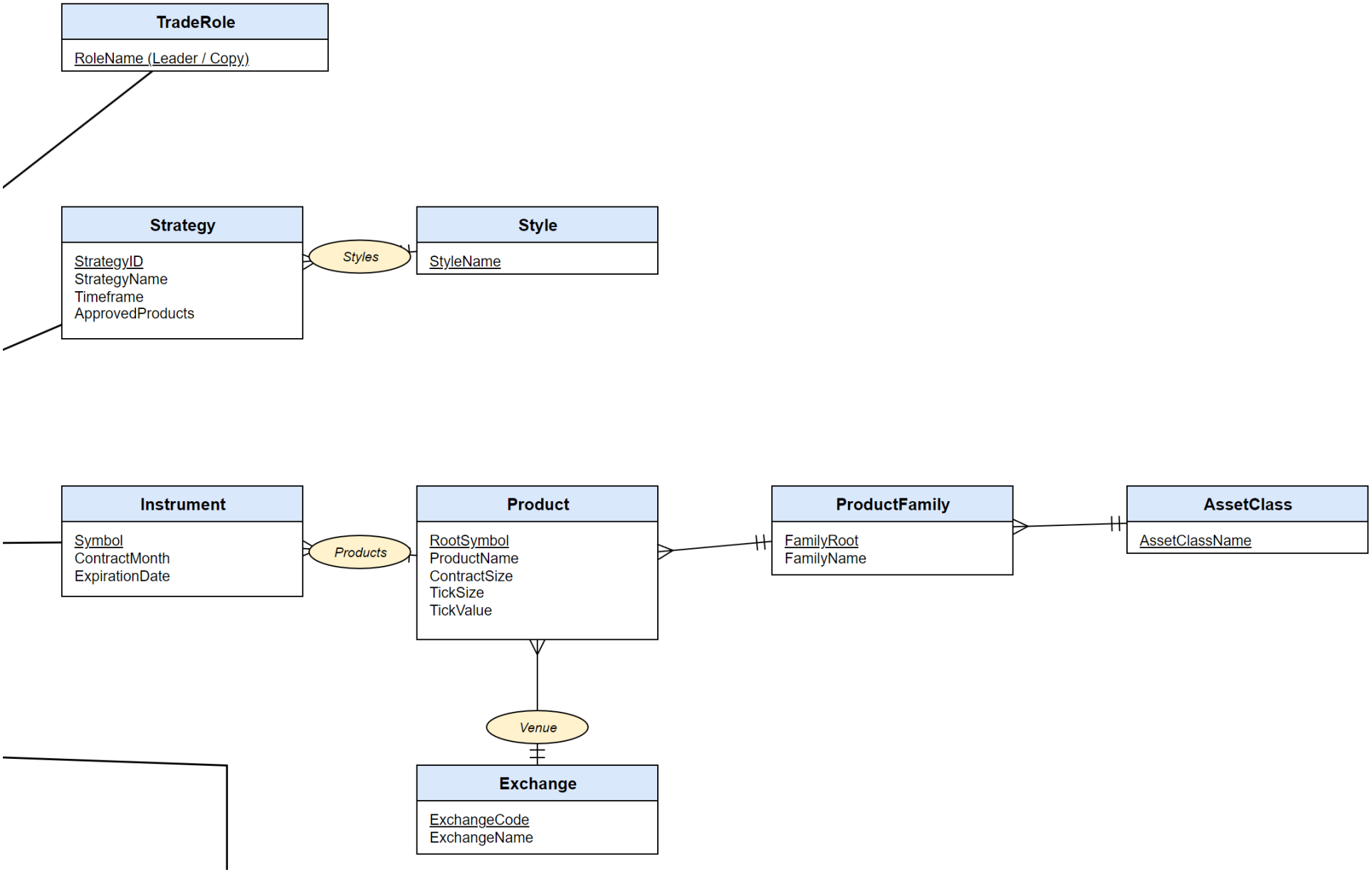
In the next milestone, I intend to rebuild the SQL load as a Talend ETL job that carries out the incremental design from Section 6.2. The job will read only the source rows beyond the high-water marks stored in etl_load_log, update the Account Lifecycle snapshot in place, and maintain the Type 2 histories of dim_account and dim_strategy when plan rules or strategy approvals change.

Appendix A. Enlarged Diagram Views

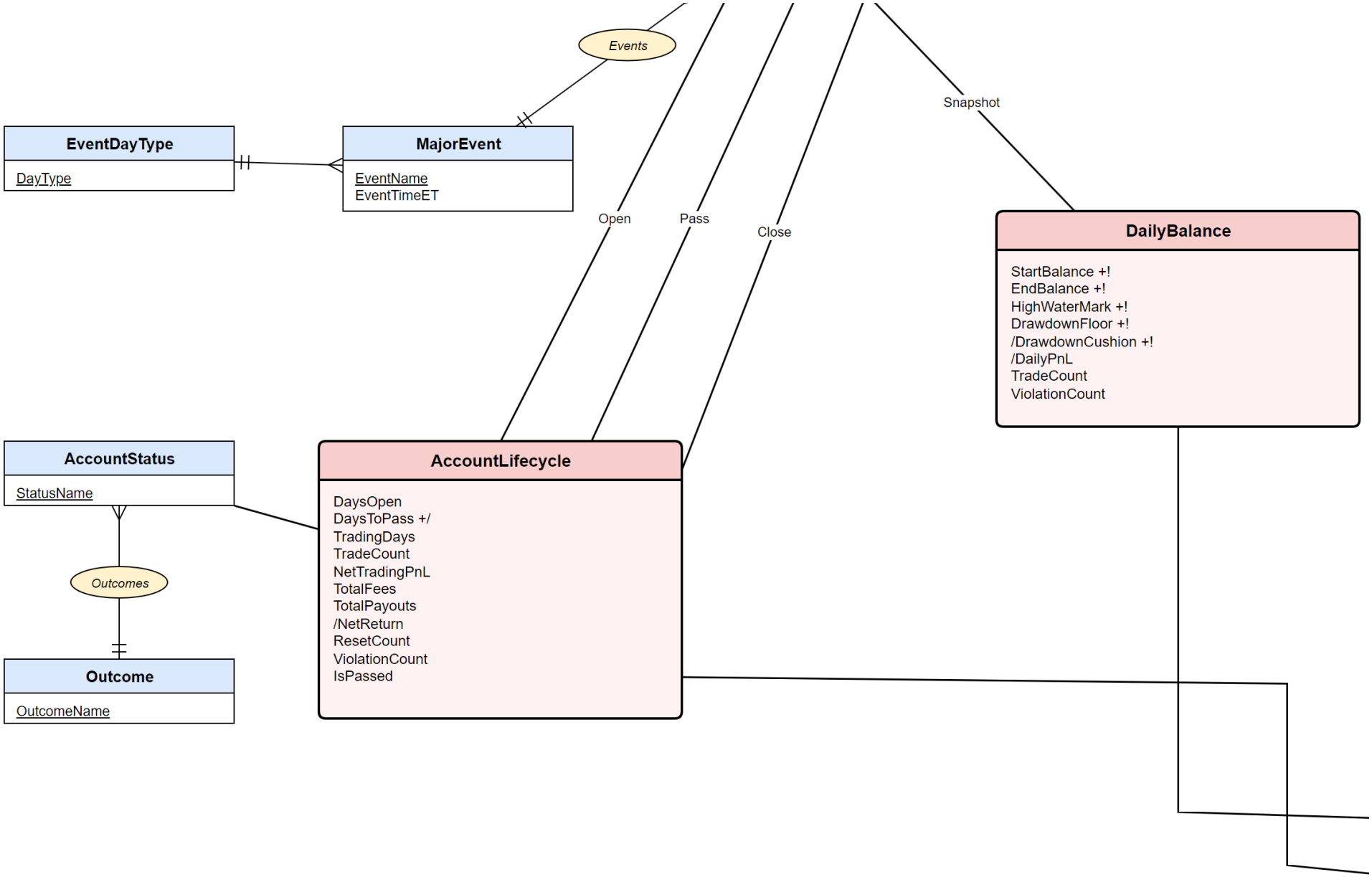
The two diagrams in F 1 and F 2 are too detailed to read comfortably at page size, so each one is repeated below in four overlapping quadrants. Every level, fact, and table appears complete in at least one view.



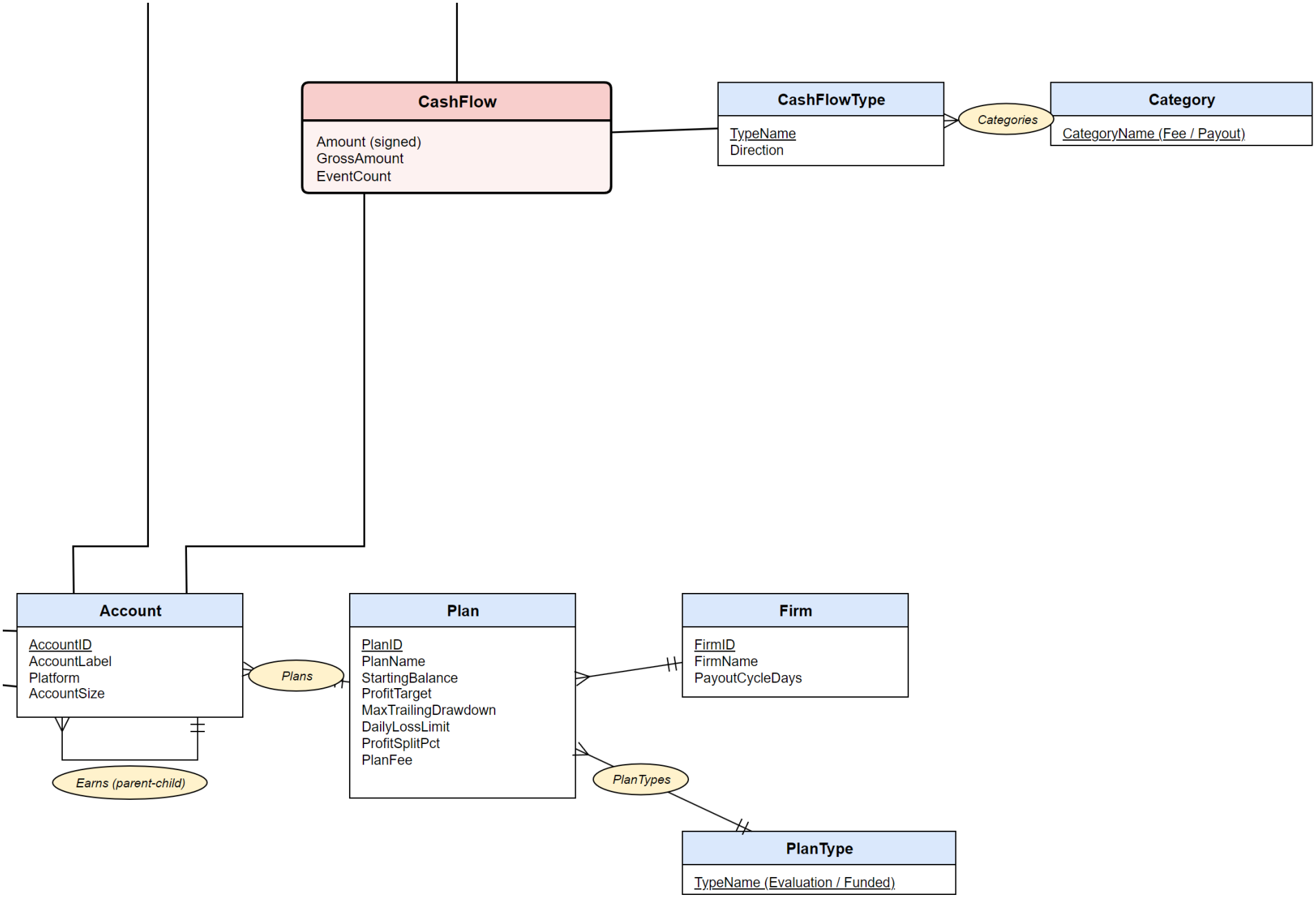
F 4. F 1 enlarged, top left: Calendar and Time of Day hierarchies and the Trade fact



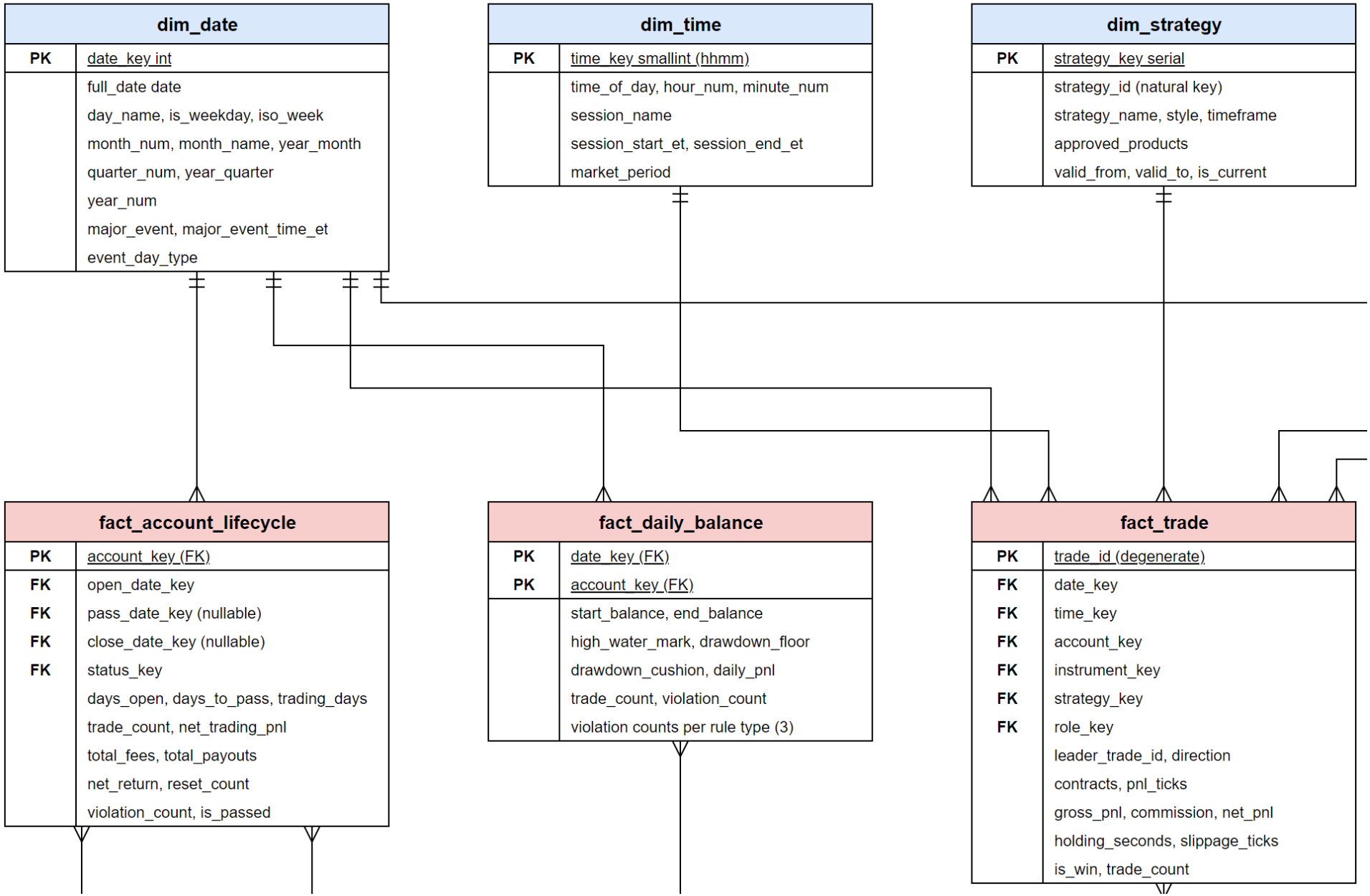
F 5. F 1 enlarged, top right: Trade Role, Strategy, and Instrument hierarchies



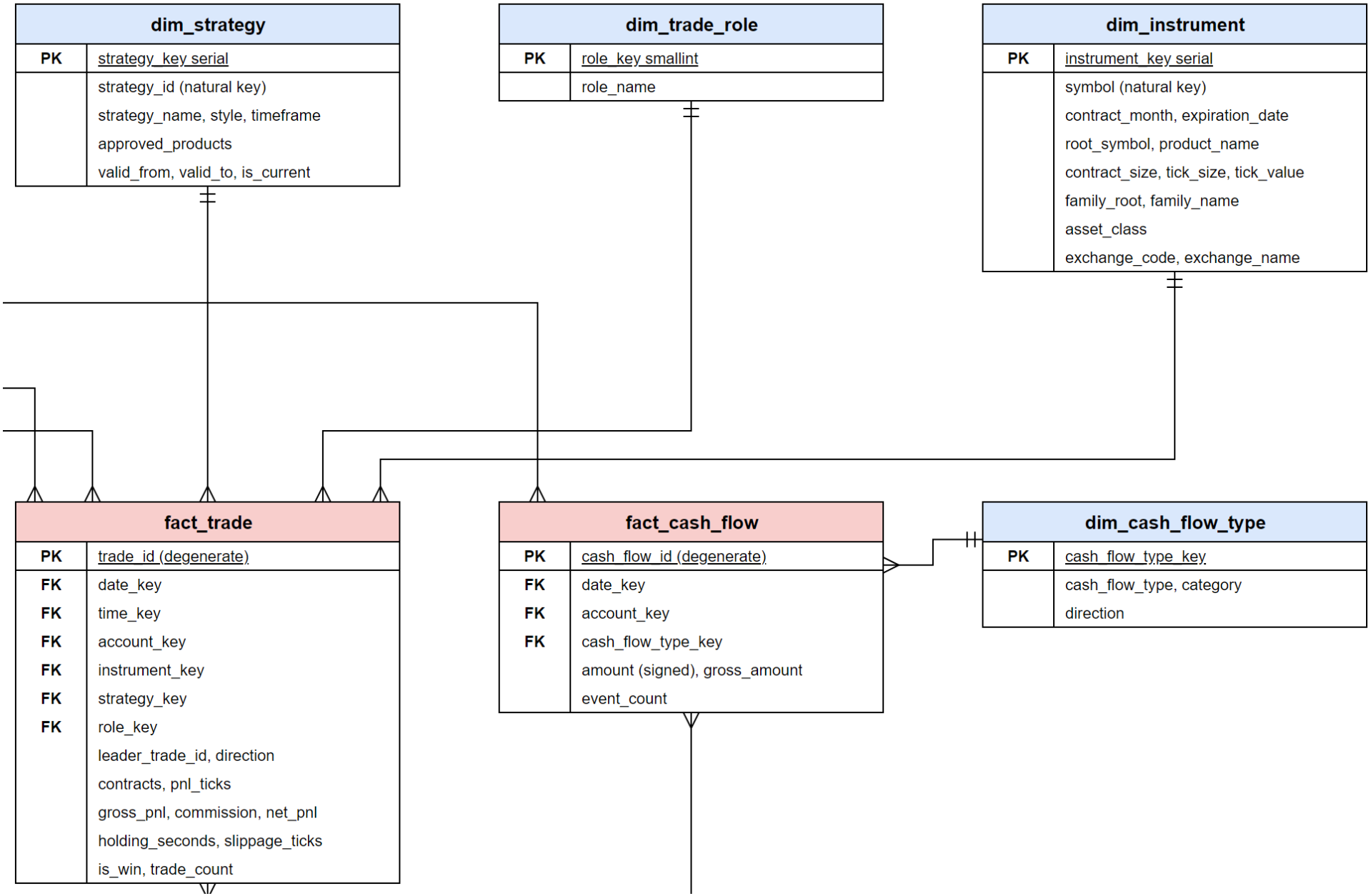
F 6. F 1 enlarged, bottom left: Events hierarchy, Account Lifecycle, Daily Balance, and Account Status



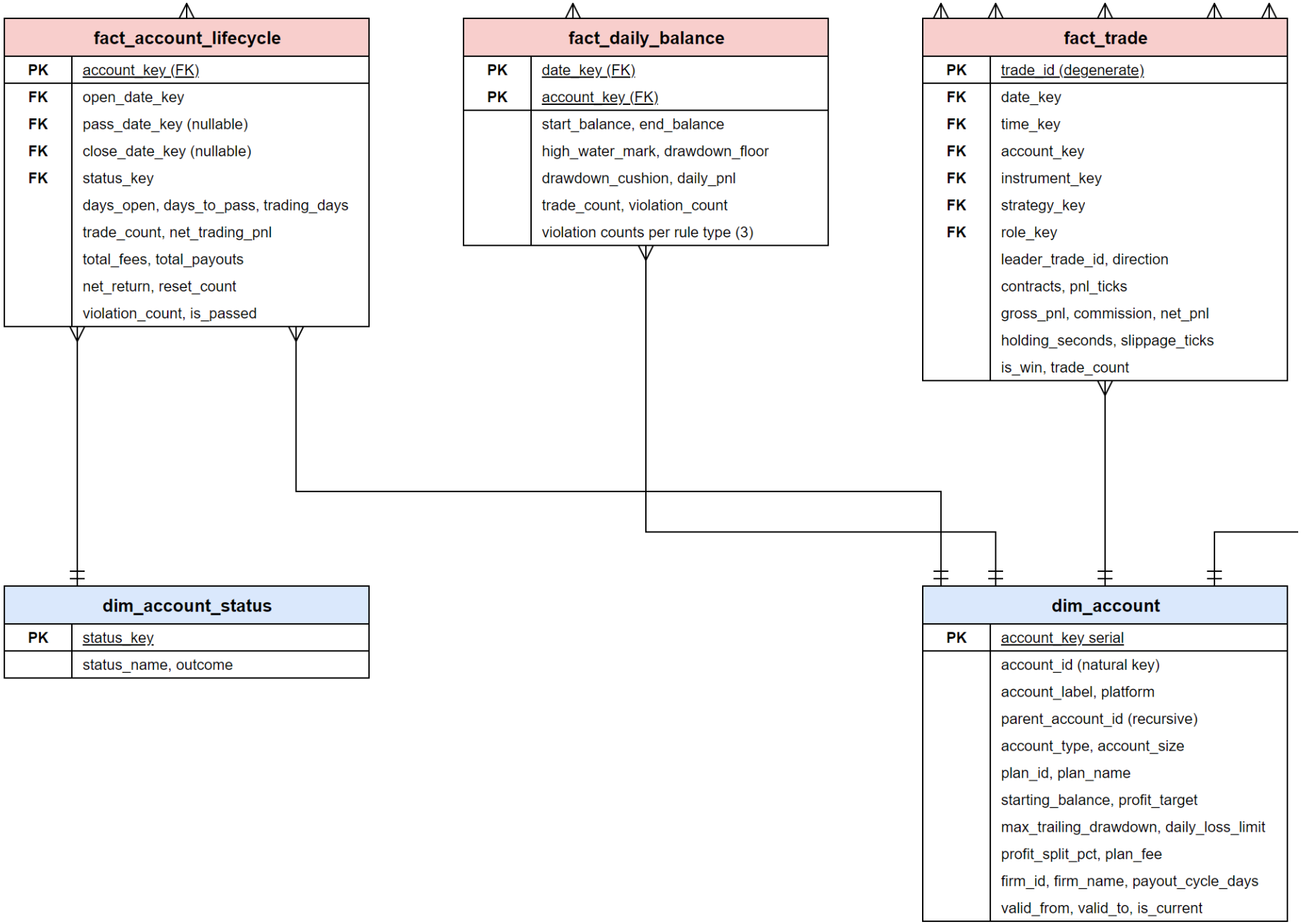
F 7. F 1 enlarged, bottom right: Cash Flow fact and the Account hierarchy



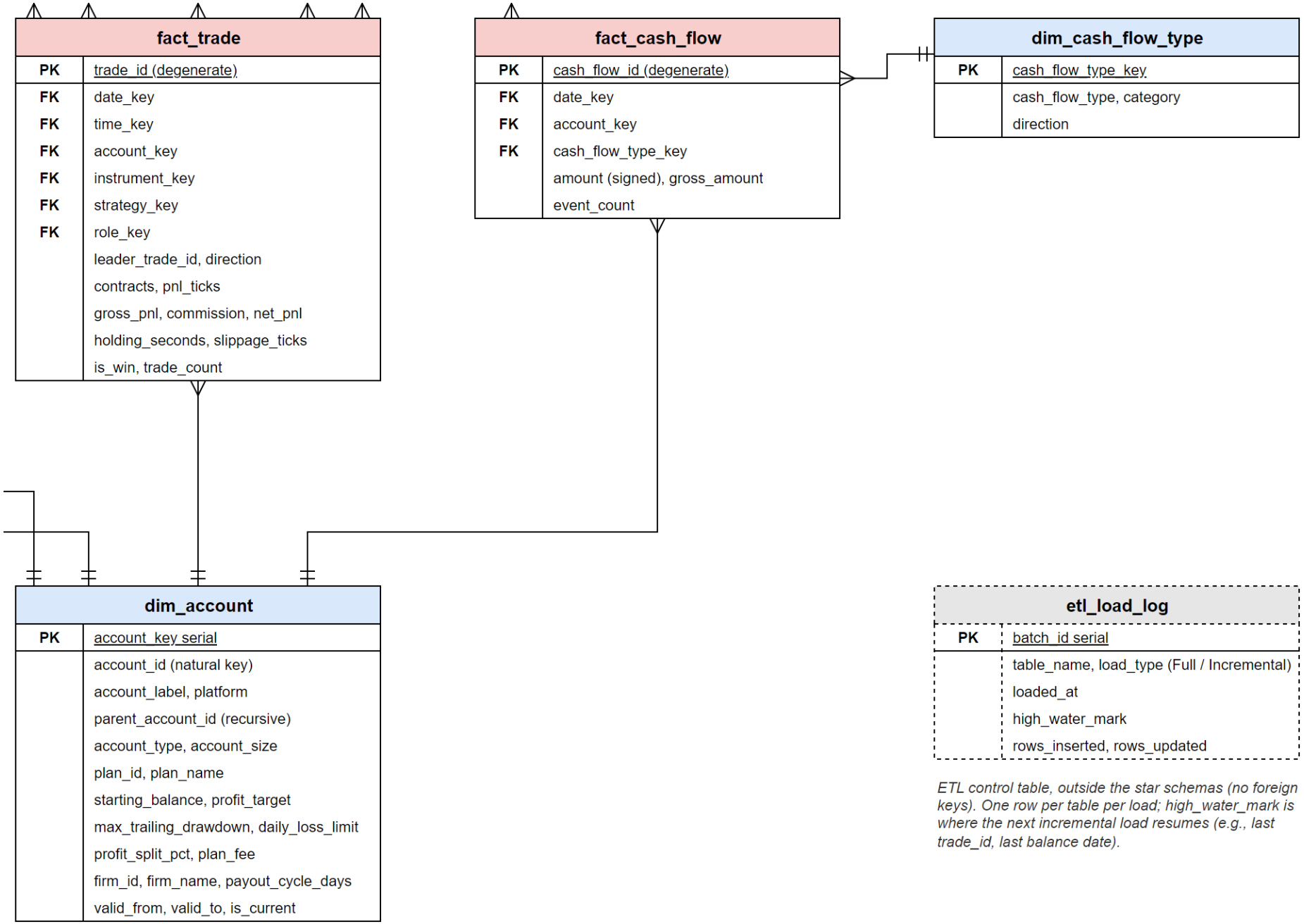
F 8. F 2 enlarged, top left: dim_date, dim_time, dim_strategy and the facts below them



F 9. F 2 enlarged, top right: dim_strategy, dim_trade_role, dim_instrument, fact_cash_flow, and dim_cash_flow_type



F 10. F 2 enlarged, bottom left: fact tables with dim_account_status and dim_account



F 11. F 2 enlarged, bottom right: fact_trade, fact_cash_flow, dim_account, and etl_load_log